

课程笔记

CS224N Lecture 6: Seq2Seq and Machine Translation

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture06>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：从语言模型到序列建模	3
1.1 课程结构概览	3
1.2 本章小结	3
2 语言模型评估：困惑度	3
2.1 什么是困惑度	3
2.2 困惑度的直觉理解	4
2.3 语言模型的历史进展	4
2.4 本章小结	5
3 RNN 的梯度问题	5
3.1 梯度消失问题	5
3.2 梯度消失的实际影响	5
3.3 梯度爆炸与梯度裁剪	5
3.4 本章小结	6
4 长短期记忆网络 (LSTM)	6
4.1 设计动机	6
4.2 LSTM 的核心组件	6
4.3 LSTM 的数学公式	6
4.4 LSTM 的图示理解	7
4.5 LSTM 为何有效：加法的秘密	7
4.6 输出门的作用	8
4.7 LSTM 也有助于缓解梯度爆炸	8
4.8 本章小结	8
5 RNN 架构扩展	8
5.1 梯度问题不仅限于 RNN	8
5.2 RNN 的多种应用	9
5.3 句子编码：RNN 作为句子表示	9
5.4 双向 RNN	9
5.5 多层（堆叠）RNN	9
5.6 本章小结	9
6 机器翻译：从统计方法到神经方法	10
6.1 机器翻译简史	10
6.2 统计机器翻译 (SMT)	10
6.3 翻译中的困难：词序重排与修饰关系	10
6.4 本章小结	10
7 序列到序列模型与神经机器翻译	11
7.1 神经机器翻译的核心思想	11
7.2 Seq2Seq 模型架构	11

7.3	训练与测试的区别	11
7.4	编码器可以是双向的	12
7.5	多层 Seq2Seq 模型	12
7.6	Seq2Seq 是通用框架	12
7.7	NMT 的历史性突破	12
7.8	本章小结	13
8	总结与延伸	13
8.1	讲者的核心总结	13
8.2	全课知识图谱	13
8.3	关键 Takeaways	14
8.4	后续展望	14
8.5	拓展阅读	14

1 引言：从语言模型到序列建模

本讲是 CS224N 第六讲，承接上一讲（Lecture 5）关于语言模型与循环神经网络的基础内容，进一步深入讨论三个核心主题：（1）语言模型的评估方法——困惑度（Perplexity）；（2）RNN 的梯度问题与 LSTM 的解决方案；（3）序列到序列模型在机器翻译中的应用。

前置知识回顾

上一讲（Lecture 5）介绍了两个独立但相关的概念：

- **语言模型**（Language Model）：预测下一个词的系统，是 NLP 中许多任务（文本生成、似然估计）的基础组件
- **循环神经网络**（RNN）：能处理任意长度序列输入的神经网络架构，在每个时间步应用相同的权重，可选择性地在每步产生输出

这两个概念常常结合使用，但 RNN 也可用于语言模型以外的其他序列任务。

1.1 课程结构概览

本讲的内容组织如下：

1. **困惑度**（Perplexity）：语言模型的标准评估指标
2. **RNN 的梯度问题**：梯度消失与梯度爆炸
3. **LSTM**：通过门控机制解决长距离依赖问题
4. **RNN 的其他应用**：双向 RNN、多层 RNN
5. **机器翻译**：从统计方法到神经方法的革命
6. **Seq2Seq 模型**：编码器-解码器架构

1.2 本章小结

本讲从语言模型的评估出发，经过 RNN 的梯度问题分析和 LSTM 的设计动机，最终引入序列到序列模型在机器翻译中的应用。这条主线贯穿了从理论分析到工程实践的完整链路。

2 语言模型评估：困惑度

2.1 什么是困惑度

语言模型的本质功能是为一段文本分配概率。评估语言模型的标准方法是：给定一段人类书写的**未见过的文本**（非训练数据），让模型逐词预测，看模型预测得有多好。

困惑度的定义

困惑度 (Perplexity) 是交叉熵的指数形式。设语言模型对测试文本 $w_1, w_2, \dots, w_T$ 的预测概率为 $P(w_t|w_1, \dots, w_{t-1})$, 则:

$$\text{Perplexity} = \prod_{t=1}^T \left(\frac{1}{P(w_t|w_1, \dots, w_{t-1})} \right)^{1/T}$$

等价地, 困惑度 = $\exp(\text{Cross Entropy})$, 即:

$$\text{Perplexity} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t|w_1, \dots, w_{t-1}) \right)$$

2.2 困惑度的直观理解

困惑度可以理解为**等效均匀选择数**: 如果困惑度为 64, 就相当于每次从 64 个等概率的选项中猜测下一个词。困惑度越低, 模型越好。

困惑度的历史由来

困惑度这一概念由 IBM 的 Fred Jelinek 在 1970–80 年代引入。当时, 符号主义 AI 的研究者对信息论和交叉熵不甚了解, Jelinek 需要一个更直观的度量来解释统计语言模型的效果。困惑度通过指数变换将交叉熵转化为“等效选择数”, 更容易被非数学背景的人理解。尽管从现代视角看, 直接使用交叉熵 (负对数似然) 更为自然, 但困惑度作为评估指标已经约定俗成。

比较困惑度时注意对数底数

困惑度的值取决于对数底数的选择。传统上使用以 2 为底的对数 (信息论传统), 现代实践中更多使用自然对数。比较不同论文的困惑度数值时, 务必确认所用的对数底数一致, 否则数值不可直接比较。

2.3 语言模型的历史进展

模型	困惑度	时期
Interpolated Kneser-Ney (5-gram)	~67	2000s
RNN + MaxEnt 混合	~51	2010s 初期
LSTM 语言模型	30–43	2015–2018
现代大语言模型	个位数	2020s

表 1: 不同时期语言模型在标准测试集上的困惑度

从 n-gram 时代的困惑度 ~67, 到早期 RNN 的 ~51, 再到 LSTM 时代的 30–43, 困惑度减半意味着交叉熵降低约 1 bit。现代大语言模型的困惑度已降至个位数, 意味着模型大多数时候能准确预测下一个词。

2.4 本章小结

困惑度是语言模型的标准评估指标，本质上是交叉熵的指数形式。数值越低代表模型越好。从 n-gram 到 LSTM 再到 Transformer，语言模型的困惑度经历了数量级的改善。

3 RNN 的梯度问题

3.1 梯度消失问题

训练 RNN 时，需要将损失通过时间反向传播（Backpropagation Through Time, BPTT）。考虑在位置 t 处的损失 $J^{(t)}(\theta)$ ，其相对于早期隐藏状态 $h^{(k)}$ 的梯度涉及一连串偏导数的乘积：

$$\frac{\partial J^{(t)}}{\partial h^{(k)}} = \frac{\partial h^{(k+1)}}{\partial h^{(k)}} \times \frac{\partial h^{(k+2)}}{\partial h^{(k+1)}} \times \cdots \times \frac{\partial J^{(t)}}{\partial h^{(t)}}$$

梯度消失的数学根源

假设忽略非线性激活函数， $h^{(t)} = W_h h^{(t-1)} + W_x x^{(t)}$ ，则：

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = W_h$$

反向传播 n 步等于计算 W_h^n 。对 W_h 进行特征值分解：

- 若所有特征值 $|\lambda_i| < 1$ ： $W_h^n \rightarrow 0$ （梯度消失）
- 若存在特征值 $|\lambda_i| > 1$ ： $W_h^n \rightarrow \infty$ （梯度爆炸）

3.2 梯度消失的实际影响

实验表明，简单 RNN 的有效上下文窗口约为 **7 个 token**——更远的信息几乎无法被学习到。这与 5-gram 语言模型相比没有实质性进步。

考虑这样一段文本：“When she tried to print her ticket, she found that the printer was out of toner. She went to the store to buy more toner. ... After installing the toner into the printer, she finally printed her ____.” 人类可以几乎 100% 确定地预测最后一个词是 “ticket”，但这依赖于 20 多个词之前的信息。简单 RNN 因梯度消失而无法建立这种长距离关联。

3.3 梯度爆炸与梯度裁剪

梯度爆炸的解决方案相对简单——**梯度裁剪**（Gradient Clipping）：

$$\hat{g} = \begin{cases} g & \text{if } \|g\| \leq \text{threshold} \\ \frac{\text{threshold}}{\|g\|} \cdot g & \text{if } \|g\| > \text{threshold} \end{cases}$$

梯度裁剪是训练神经网络的必备技巧

梯度裁剪虽然是一个“粗暴的 hack”，但它非常有效且几乎是必需的。阈值通常设为 5、10 或 20。梯度裁剪保持了梯度的方向，只限制其大小，避免了过大步长导致的训练不稳定。在训练任何深度模型时，都应该考虑使用梯度裁剪。

3.4 本章小结

- **梯度消失**：梯度信号在长序列中指数级衰减，导致模型无法学习长距离依赖（有效窗口仅 ~ 7 个 token）
- **梯度爆炸**：梯度信号指数级增长，导致参数更新失控，可通过梯度裁剪解决
- 梯度消失是更根本的问题，需要从**架构层面**解决——这正是 LSTM 的动机

4 长短期记忆网络 (LSTM)

4.1 设计动机

简单 RNN 的核心问题在于：隐藏状态在每个时间步被**完全改写**（通过矩阵乘法），这使得信息很难被保留。我们需要一种架构，使得信息可以被**选择性地添加和遗忘**，而非每步全部覆盖。

LSTM 的历史

LSTM (Long Short-Term Memory) 由 Hochreiter 和 Schmidhuber 于 1997 年提出。名称的含义是“长期的短期记忆”——人类的短期记忆可以保持相当长时间（几分钟到几十分钟），而简单 RNN 的“短期记忆”仅约 7 个 token。LSTM 旨在延长这一有效记忆窗口。值得注意的是，现代常用 LSTM 的一个关键组件——遗忘门——并不在 1997 年的原始论文中，而是由 Gers 和 Schmidhuber 在 2000 年的后续工作中引入的。Schmidhuber 的学生 Alex Graves 后来将 LSTM 带到 Hinton 的实验室，最终经由 Google 在 2014–2016 年间大规模部署，LSTM 才成为主流。

4.2 LSTM 的核心组件

LSTM 在每个时间步维护两个状态向量：

- **隐藏状态** (Hidden State) $h^{(t)}$ ：用于预测输出和传递给下一步
- **细胞状态** (Cell State) $c^{(t)}$ ：作为长期记忆的载体

信息的流动由三个门 (Gate) 控制，每个门是一个与隐藏状态同维度的向量，值在 $[0, 1]$ 之间：

4.3 LSTM 的数学公式

第一步：计算三个门

$$f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$$

$$i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i)$$

$$o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o)$$

各符号说明：

- $f^{(t)}$ ：**遗忘门** (Forget Gate)，控制保留多少旧的细胞状态。 σ 为 sigmoid 函数，输出在 $[0, 1]$ 之间

- $i^{(t)}$: **输入门** (Input Gate), 控制写入多少新信息到细胞
- $o^{(t)}$: **输出门** (Output Gate), 控制从细胞中读出多少信息到隐藏状态
- W_f, W_i, W_o : 隐藏状态到各门的权重矩阵
- U_f, U_i, U_o : 输入到各门的权重矩阵
- b_f, b_i, b_o : 偏置项

第二步：计算候选细胞内容

$$\tilde{c}^{(t)} = \tanh(W_c h^{(t-1)} + U_c x^{(t)} + b_c)$$

这个候选内容的计算形式与简单 RNN 完全相同，只是使用了专用的权重矩阵 W_c 和 U_c 。

第三步：更新细胞状态

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

- $\odot$: Hadamard 积 (逐元素乘法)
- $f^{(t)} \odot c^{(t-1)}$: 遗忘门控制保留多少旧记忆
- $i^{(t)} \odot \tilde{c}^{(t)}$: 输入门控制写入多少新信息

第四步：计算隐藏状态

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

输出门控制从细胞状态中“读出”多少信息到隐藏状态。 $\tanh$ 将细胞状态的值压缩到 $[-1, 1]$ 。

遗忘门实际上是“记忆门”

Manning 指出，“遗忘门”的命名容易引起误解。当 $f^{(t)} = 1$ 时，完全**保留**旧记忆；当 $f^{(t)} = 0$ 时，完全**遗忘**。因此，将其理解为“记忆门” (Remember Gate) 更符合直觉——它的值越大，保留得越多。

4.4 LSTM 的图示理解

4.5 LSTM 为何有效：加法的秘密

加法操作是 LSTM 的核心秘密

简单 RNN 的隐藏状态更新是纯**乘法**的 ($h^{(t)} = \sigma(W_h h^{(t-1)} + \dots)$)，信息在乘法操作中容易消失或爆炸。而 LSTM 的细胞状态更新包含一个**加法**操作：

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

这个加号是关键——它使得信息可以沿细胞状态直接流动，无需经过矩阵乘法。当 $f^{(t)} = 1$ 且 $i^{(t)} = 0$ 时， $c^{(t)} = c^{(t-1)}$ ，信息可以无损地线性传递。这从根本上解决了梯度消失问题。

⁰Slides 第 23 页。来源：<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

⁰Slides 第 24 页。来源：Chris Olah 博客。

这种加法更新机制与人类记忆的工作方式类似：我们通过**添加**新记忆来更新认知，而非每次完全重写。最初的 LSTM (1997) 甚至没有遗忘门，是纯粹的加法机制。但纯加法在长序列上会累积过多信息导致功能失调，因此后来加入了遗忘门来选择性地清除部分记忆。

4.6 输出门的作用

隐藏状态 $h^{(t)}$ 承担双重职责：(1) 用于预测当前位置的下一个词；(2) 存储可能对未来预测有用的上下文信息。输出门使得 LSTM 可以将这两个功能分离——细胞状态存储所有可能有用的信息，而输出门选择性地将当前需要的部分传递到隐藏状态用于输出预测。

例如，如果之前的文本提到了“the King of Prussia”，这个信息需要被记住以供未来使用，但在预测当前紧邻的下一个词时可能不相关。输出门可以让这部分信息留在细胞状态中而不暴露到隐藏状态。

4.7 LSTM 也有助于缓解梯度爆炸

LSTM 的加法更新不仅解决了梯度消失，也有助于缓解梯度爆炸。因为梯度不再需要经过连续的矩阵乘法链——加法操作提供了一条“梯度高速公路”，使梯度信号可以更稳定地流动。

4.8 本章小结

- LSTM 通过引入**细胞状态**和三个门（遗忘门、输入门、输出门），实现了信息的选择性保留和更新
- **加法操作**是 LSTM 的核心设计——它允许梯度沿细胞状态直接流动，解决了梯度消失问题
- 当遗忘门为 1、输入门为 0 时，信息可以无损地沿时间线性传递
- LSTM 使有效上下文窗口从 ~ 7 个 token 扩展到数十甚至数百个 token

5 RNN 架构扩展

5.1 梯度问题不仅限于 RNN

梯度消失和爆炸并非 RNN 独有的问题。在任何深层网络中，梯度都需要经过多层传递，同样可能出现指数级的衰减或增长。这也是为什么早期深层神经网络难以训练的原因。

残差连接：深层网络的类似解决方案

残差网络 (ResNet, He et al., 2015) 通过在层间添加**恒等映射** (skip connection) 来解决深层前馈网络的梯度问题： $\mathcal{F}(x) + x$ 。这与 LSTM 的思路异曲同工——都是通过**加法**提供信息直通的路径。相关的变体包括：

- **DenseNet**：每一层与所有后续层都直接连接
- **Highway Network** (Srivastava et al., Schmidhuber 团队)：在残差连接上增加门控，类似 LSTM 的门控思路

5.2 RNN 的多种应用

一旦有了 RNN (尤其是 LSTM)，它就可以用于各种序列相关的 NLP 任务：

- **序列标注**：词性标注 (POS Tagging)、命名实体识别 (NER) —— 在每个位置输出一个标签
- **句子编码**：情感分类等——将整个句子编码为一个固定长度的向量，用于分类
- **条件生成**：机器翻译、语音识别、文本摘要——在给定输入条件下生成输出序列

5.3 句子编码：RNN 作为句子表示

对于句子级别的分类任务（如情感分析），可以用 RNN 处理整个句子，然后使用得到的表示进行分类。最简单的方法是取最后一步的隐藏状态 $h^{(T)}$ ，但实践中更好的做法是对所有时间步的隐藏状态取均值或逐元素最大值。

5.4 双向 RNN

单向 RNN 在每个位置只能看到左侧的上下文。但对于很多任务（如序列标注），我们希望每个位置的表示能综合左右两侧的信息。

双向 RNN (Bidirectional RNN)

- 分别运行一个**前向** LSTM（从左到右）和一个**后向** LSTM（从右到左）
- 在每个位置，将两个方向的隐藏状态**拼接**： $h^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$
- 适用于**编码**任务（分类、标注等），但**不能**用于语言生成（因为后向 RNN 需要看到未来的词）

5.5 多层（堆叠）RNN

将多层 RNN 堆叠可以提升模型的表示能力，类似于深层前馈网络通过多层实现更高级的特征抽象。

RNN 时代的“深度”有限

与现代 Transformer 动辄几十甚至上百层不同，RNN 时代人们通常只使用 2-3 层。从 1 层到 2 层的提升很明显，但继续增加层数的收益递减。Manning 指出，这一状况在 Transformer 时代已完全改变——现代语言模型使用非常深的 Transformer 网络。

5.6 本章小结

- 梯度问题不仅限于 RNN，深层前馈网络的解决方案（残差连接）与 LSTM 的加法更新思路一脉相承
- **双向 RNN** 提供了每个位置的完整上下文表示，适合编码任务但不能用于生成
- **多层 RNN** 通过堆叠增强表示能力，RNN 时代通常用 2-3 层
- LSTM 在 2013-2015 年间成为几乎所有 NLP 任务的主导方法

6 机器翻译：从统计方法到神经方法

6.1 机器翻译简史

机器翻译是 NLP 领域最古老的任务之一，也是推动整个领域发展的核心动力。

机器翻译的起源

机器翻译的研究可追溯到 1950 年代初——比人工智能作为一个学科诞生还要早。二战期间计算机被用于两件事：（1）计算火炮弹道表；（2）密码破译。冷战期间，人们想到：如果计算机能破解密码，也许也能“破解”语言之间的翻译。大量资金投入这一方向，但在 1950 年代完全失败了——当时人们对语言结构几乎一无所知（Chomsky 的形式语言理论尚未建立），计算机的能力也极其有限。

6.2 统计机器翻译 (SMT)

机器翻译在 1990–2000 年代重新焕发生机，这一时期的主流方法是**统计机器翻译** (Statistical Machine Translation, SMT)。

SMT 的核心思想是利用贝叶斯公式将翻译问题分解为两个子问题：

$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)} \propto P(x|y) \cdot P(y)$$

- $P(x|y)$: **翻译模型**——词与词之间如何对应翻译（相对简单的词典级模型）
- $P(y)$: **语言模型**——目标语言中什么样的句子是自然的（提供语法和流畅度约束）

SMT 系统的工程复杂度

Google 的统计机器翻译系统经过十多年、数百名工程师的开发，包含数百万行代码、大量针对特定语言对的手工规则和 hack。每增加一个语言对，都需要重复大量工程工作。系统需要维护短语表、重排序模型、平滑算法等众多独立设计的子组件。

6.3 翻译中的困难：词序重排与修饰关系

Manning 展示了一个经典的中译英示例：“1519 年 600 名西班牙人在墨西哥登陆，去征服几百万人口的阿兹特克帝国，初次交锋他们损兵三分之二。” Google Translate 在 2009–2015 年间多次翻译这个句子，始终无法正确处理“几百万人口的阿兹特克帝国”这一修饰关系，错误地将“几百万人”理解为征服的主语。

这一例子清楚地展示了 SMT 的根本局限：它无法有效捕捉跨语言的**句法结构差异**和**长距离修饰关系**。

6.4 本章小结

- 机器翻译是 NLP 最古老的任务之一，经历了规则方法（1950s）、统计方法（1990s–2010s）和神经方法（2014–）三个阶段
- 统计机器翻译通过贝叶斯分解将问题拆解为翻译模型和语言模型，但工程极其复杂
- 不同语言之间的词序重排、修饰关系差异、功能词增减等问题，使翻译远比“逐词替换”复杂

7 序列到序列模型与神经机器翻译

7.1 神经机器翻译的核心思想

2014 年, Sutskever et al. 提出了用**单一的端到端神经网络**来解决机器翻译问题——这就是**神经机器翻译** (Neural Machine Translation, NMT)。

NMT 的革命性简化

相比于 SMT 数百万行代码和众多手工设计的子组件, NMT 只需要一个相对简单的 encoder-decoder 神经网络, 直接学习 $P(y|x)$ 。所有组件的参数通过同一个损失函数端到端联合优化——这正是深度学习“端到端学习”范式的威力。

7.2 Seq2Seq 模型架构

Seq2Seq 模型由两个 RNN (通常是 LSTM) 组成:

编码器 (Encoder):

- 逐词处理源语言句子 $x_1, x_2, \dots, x_n$
- 不产生输出, 只积累隐藏状态
- 最终隐藏状态 h_{enc} 编码了整个源句子的信息

解码器 (Decoder):

- 以编码器的最终隐藏状态作为初始状态
- 作为**条件语言模型**, 逐步生成目标语言的词
- 从 <START> 标记开始, 每步将生成的词作为下一步的输入
- 直到生成 <END> 标记为止

条件语言模型

解码器本质上是一个**条件语言模型**——它不是从无到有生成文本, 而是在给定源句子编码的条件下生成翻译。训练目标是最大化:

$$P(y_1, y_2, \dots, y_m | x_1, x_2, \dots, x_n) = \prod_{t=1}^m P(y_t | y_1, \dots, y_{t-1}, x_1, \dots, x_n)$$

7.3 训练与测试的区别

训练时使用真实目标词, 测试时使用模型生成的词

训练时 (Teacher Forcing): 解码器在每步接收的输入是**真实的目标词** (ground truth), 损失是对每个位置的预测概率取负对数似然。

测试时: 解码器在每步接收的输入是**自己上一步生成的词** (argmax 或采样)。这意味着测试时的错误可能累积——一个错误的词可能导致后续所有翻译偏离。这种训练与测试之间的不一致被称为 **exposure bias**。

7.4 编码器可以是双向的

Manning 提到，编码器可以使用**双向 LSTM**，因为编码阶段可以看到源句子的全部内容。但解码器**不能**是双向的，因为生成过程是从左到右逐步进行的。在 Sutskever et al. (2014) 的原始论文中，编码器是单向的；后续工作（如 Luong et al., 2015）表明双向编码器能带来提升。

7.5 多层 Seq2Seq 模型

在实际部署中，机器翻译系统使用多层堆叠的 LSTM 作为编码器和解码器。这是 LSTM 多层堆叠确实能带来显著收益的少数场景之一。

信息瓶颈问题

在基础的 Seq2Seq 模型中，整个源句子的信息被压缩到编码器最后一步的隐藏状态向量中。这个固定大小的向量需要承载任意长度句子的全部语义信息，形成了**信息瓶颈**（Information Bottleneck）。这一问题催生了后续的**注意力机制**（Attention Mechanism）——下一讲的主题。

7.6 Seq2Seq 是通用框架

Seq2Seq 架构的适用范围远超机器翻译：

- **文本摘要**：长文本 → 短文本
- **对话系统**：历史对话 → 下一轮回复
- **句法分析**：输入文本 → 序列化的解析树
- **代码生成**：自然语言描述 → 编程语言代码

甚至在后来的 Transformer 时代，编码器-解码器的框架思想依然是核心架构范式之一（如 T5 模型）。

7.7 NMT 的历史性突破

NMT 的成功是深度学习在 NLP 领域的第一个重大突破：

- **2014 年**：Sutskever et al. (Google) 首次展示 LSTM-based NMT 系统
- **2016 年**：仅两年后，Google Translate 全面切换到 NMT 系统
- 翻译质量的提升**肉眼可见**——用户在 Google 未公开宣布时就注意到了翻译质量的飞跃
- 此后所有主要公司（Microsoft、Facebook/Meta、百度、腾讯等）都迅速转向 NMT

NMT 为何如此成功

1. **端到端训练**：所有参数针对翻译目标联合优化，无需手工设计子组件
2. **分布式表示**：词向量和隐藏状态能捕捉丰富的语义和语法信息
3. **上下文建模**：LSTM 能在一定程度上处理长距离依赖和词序重排
4. **简洁性**：相比 SMT 数百万行代码，NMT 系统代码量小几个数量级

7.8 本章小结

- 神经机器翻译使用 Seq2Seq (编码器-解码器) 架构, 以单一端到端网络直接学习翻译映射
- 编码器将源句子压缩为固定长度向量, 解码器基于此向量生成目标句子
- 训练使用 Teacher Forcing, 测试时使用自回归生成
- NMT 在 2014-2016 年间取得了革命性成功, 彻底取代了 SMT
- Seq2Seq 框架具有通用性, 适用于各种序列转换任务
- 基础 Seq2Seq 存在信息瓶颈问题, 将在下一讲通过注意力机制解决

8 总结与延伸

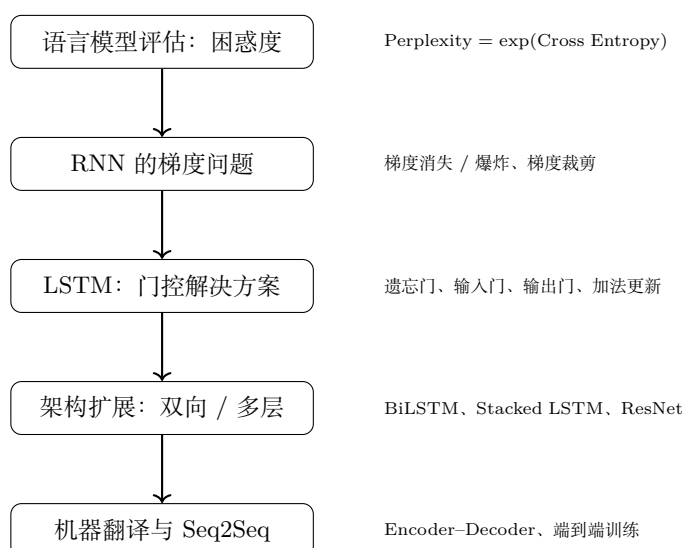
8.1 讲者的核心总结

Chris Manning 在课程结尾总结了四个核心要点:

1. **LSTM 很强大**: 如果需要使用 RNN, 应该优先选择 LSTM
2. **梯度裁剪**: 训练任何深度模型时都应该使用梯度裁剪
3. **双向性**: 在编码任务中尽可能使用双向 RNN
4. **Encoder-Decoder NMT 效果优异**: Seq2Seq 架构在机器翻译上取得了革命性突破

8.2 全课知识图谱

本课建立了一条从问题分析到解决方案再到应用的完整链路:



⁰Slides 第 56 页 (最后一页)。

8.3 关键 Takeaways

五条核心原则

1. **梯度消失是架构问题**: 简单 RNN 的乘法更新导致信息在长序列中指数衰减, 需要从架构层面 (加法更新) 解决
2. **加法是信息保存的关键**: LSTM 的细胞状态加法更新、ResNet 的残差连接, 本质上都是通过加法提供梯度直通路
3. **门控是灵活性的来源**: LSTM 的三个门使网络能学会何时记忆、何时遗忘、何时输出, 而非依赖固定规则
4. **端到端训练的威力**: NMT 的成功表明, 一个简单的端到端模型可以超越数十年精心设计的复杂系统
5. **Seq2Seq 是通用框架**: 编码器-解码器架构不仅用于翻译, 是所有序列转换任务的基础范式

8.4 后续展望

本讲的 Seq2Seq 模型存在一个关键限制: 编码器将整个源句子压缩到一个固定长度的向量中, 形成信息瓶颈。下一讲将介绍**注意力机制** (Attention Mechanism), 它允许解码器在每一步“回看”编码器的所有隐藏状态, 从而打破这一瓶颈。注意力机制不仅大幅提升了 NMT 的性能, 更为后来的 **Transformer** 架构 (“Attention is All You Need”) 奠定了基础。

8.5 拓展阅读

- Hochreiter & Schmidhuber, “Long Short-Term Memory” (1997): <https://www.bioinf.jku.at/publications/older/2604.pdf>
- Chris Olah, “Understanding LSTMs”: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- Sutskever, Vinyals & Le, “Sequence to Sequence Learning with Neural Networks” (2014): <https://arxiv.org/abs/1409.3215>
- Luong, Pham & Manning, “Effective Approaches to Attention-based Neural Machine Translation” (2015): <https://arxiv.org/abs/1508.04025>
- He et al., “Deep Residual Learning for Image Recognition” (2015): <https://arxiv.org/abs/1512.03385>
- Pascanu, Mikolov & Bengio, “On the difficulty of training recurrent neural networks” (2013): <https://arxiv.org/abs/1211.5063>
- The New York Times: “The Great A.I. Awakening” (Google NMT deployment): <https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html>